

Software Design & Analysis

Complete Exam Study Guide — With Full UML Diagrams

Class Diagram | Sequence Diagram | Coupling & Cohesion | SOLID Principles | Activity Diagram | State Machine Diagram

This guide contains detailed explanations, real-world scenarios, complete hand-drawn UML diagrams, code examples, comparison tables, and exam tips for every topic on your Software Design & Analysis syllabus.

Chapter 1 — UML Class Diagram

A Class Diagram is a static structural diagram in UML (Unified Modeling Language). It shows the system's classes, their attributes (data), their operations (methods), and the relationships between them. It is the blueprint of your software — drawn before a single line of code is written.

Class diagrams answer the question: What does the system contain and how are its parts related?

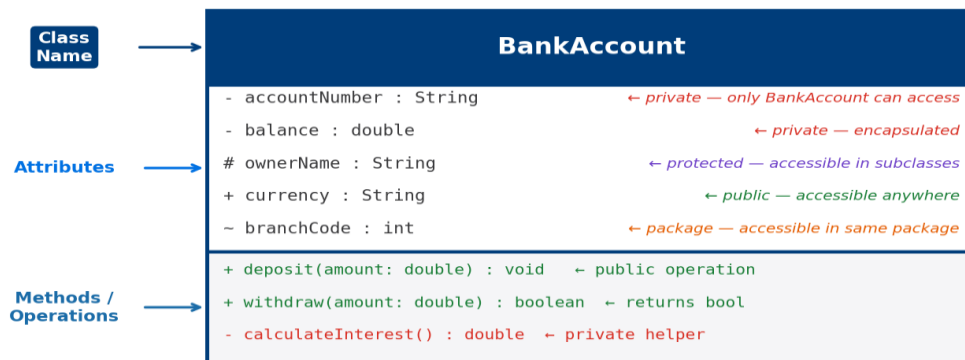
1.1 Structure of a Class — The Three-Compartment Box

Every class in a class diagram is drawn as a rectangle divided into three horizontal compartments:

- **Top compartment — Class Name, written in bold and centred (e.g., BankAccount)**
- **Middle compartment — Attributes (properties/data fields), each with visibility symbol, name, and type**
- **Bottom compartment — Operations (methods/behaviour), each with visibility, name, parameters, and return type**

Below is a fully annotated class box showing every element:

UML Class Notation — Anatomy of a Class Box



1.2 Visibility Modifiers

Before every attribute or method name, a visibility symbol declares who can access it:

Symbol	Visibility	Meaning / Access Rule
+	Public	Accessible from ANY class in the entire system
-	Private	ONLY accessible within this class itself
#	Protected	Accessible in this class AND all subclasses
~	Package	Accessible by all classes in the same package

1.3 Multiplicity

Multiplicity is written at both ends of a relationship line. It defines how many instances of one class can relate to a single instance of another class. This is critical for database and system design.

Notation	Meaning
1	Exactly one — mandatory, single relationship
0..1	Zero or one — optional (may or may not exist)
* or 0..*	Zero or more — unlimited optional instances
1..*	One or more — at least one must exist
3..5	Between 3 and 5 (range notation)

EXAM TIP: A Customer can have many BankAccounts, but each account belongs to exactly one Customer. Write '1' at the Customer end and '0..*' at the Account end of the line.

1.4 The Six Relationship Types — Explained Fully

This is the single most-tested area of class diagrams. Memorise each notation and when to use it.

1. Association — Solid Line

The most general relationship. Two classes are aware of each other and interact. Named with a verb phrase describing the relationship. No ownership is implied.

Real-World Example: A Student ATTENDS a Course. A Driver DRIVES a Car. A Teacher TEACHES a Class. All of these are associations — the objects interact, but neither owns the other.

Notation: A plain solid line between the two classes. May have an arrow to show direction (navigability). Multiplicity is written at each end.

2. Aggregation — Hollow Diamond (weak 'has-a')

A specialised form of association representing a whole-part relationship. The PART can exist independently of the WHOLE. The hollow (unfilled) diamond is placed at the WHOLE end.

Real-World Example: A University HAS Departments. If the University closes, the Professors still exist as people. A Playlist HAS Songs — delete the playlist, the songs still exist on your device.

Key test: Can the part survive without the whole? YES = Aggregation.

3. Composition — Filled Diamond (strong 'has-a')

A STRONGER whole-part relationship. The PART CANNOT exist without the WHOLE. When the whole is destroyed, all its parts are automatically destroyed too. The filled (solid, black) diamond is at the WHOLE end.

Real-World Example: A House has Rooms. Demolish the house — the rooms cease to exist. A Human Body has a Heart — the heart cannot exist (functionally) without the body. An Order has OrderItems — delete the order and its items disappear.

Key test: Does the part's existence depend entirely on the whole? YES = Composition.

4. Inheritance / Generalisation — Hollow Triangle Arrow

Also called an 'is-a' relationship. A subclass (child) inherits ALL attributes and methods of its superclass (parent) and can add its own or override them. The hollow triangular arrowhead points TOWARD the parent class.

Real-World Example: Dog IS-AN Animal. Cat IS-AN Animal. SavingsAccount IS-A BankAccount. Manager IS-AN Employee. All properties and behaviours of the parent are automatically available in the child.

Key test: Can you say 'X is a type of Y'? YES = Inheritance.

5. Realization / Implementation — Dashed Line + Hollow Triangle

A class implementing an interface. The interface specifies WHAT must be done (the contract). The implementing class specifies HOW it is done. The dashed line with hollow arrowhead points toward the interface.

Real-World Example: IPayment interface declares pay() and refund(). CreditCardPayment implements them by charging a card. PayPalPayment implements them by calling the PayPal API. Both fulfill the same contract, differently.

6. Dependency — Dashed Arrow

The weakest relationship. One class temporarily uses another — typically as a method parameter, local variable, or return type. A change to the target class may require a change to the source class.

Real-World Example: A Report class uses a DatabaseConnection to fetch data but doesn't store a reference to it permanently. The connection is created, used briefly, and discarded.

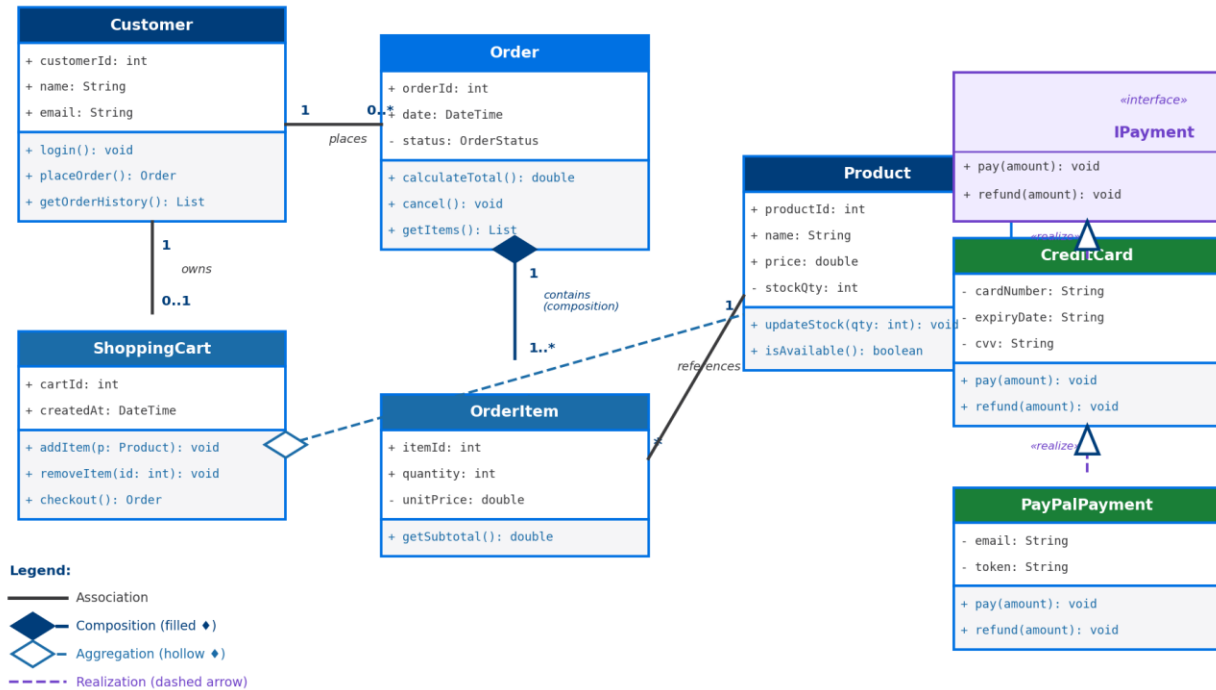
1.5 Relationship Quick-Reference Table

Relationship	Symbol	Real Example	Key Question
Association	Solid line	Student attends Course	Do they interact?
Aggregation	Hollow diamond	Playlist has Songs	Can part survive alone?
Composition	Filled diamond	Order has OrderItems	Destroyed together?
Inheritance	Hollow triangle arrow	Dog is-an Animal	Is a type of?
Realization	Dashed + hollow arrow	PayPal implements IPayment	Fulfills a contract?
Dependency	Dashed arrow only	Report uses DBConnection	Temporarily uses?

1.6 Full Class Diagram — E-Commerce System

This diagram models a complete e-commerce platform (similar to Amazon or Daraz). Study every relationship carefully — it includes all six types.

UML Class Diagram — E-Commerce System



Reading the Diagram

- Customer —1—0..*— Order: Association. One customer places many orders, each order belongs to one customer.
- Order ◆—1..*— OrderItem: COMPOSITION (filled diamond). OrderItems cannot exist without their parent Order.
- ShoppingCart ◇— Product: AGGREGATION (hollow diamond). Cart references products; products exist independently.
- CreditCard - - - ▷ IPayment: REALIZATION. CreditCard implements the IPayment interface.
- PayPalPayment - - - ▷ IPayment: REALIZATION. A different implementation of the same interface.

EXAM TIP: The most common exam trap: Students confuse Aggregation and Composition. REMEMBER — filled diamond (solid black) = composition = 'dies with parent'. Hollow diamond = aggregation = 'survives parent'.

Chapter 2 — UML Sequence Diagram

A Sequence Diagram is a behavioral UML diagram. While class diagrams show structure (what exists), sequence diagrams show dynamics (what happens). They capture the exact order of messages exchanged between objects during one specific scenario or use case.

Core question answered: Who sends what message to whom, and in what order?

2.1 Core Components

Component	Symbol / Notation	Description
Actor	Stick figure	External entity (user or external system) that initiates the interaction. Lives outside the system boundary.
Lifeline	Box + vertical dashed line	Represents a participant (object/class). The dashed line shows its existence across time.
Activation Bar	Thin solid rectangle on lifeline	Shows the period when an object is actively executing — waiting for or processing a call.
Synchronous Message	Solid line + filled arrowhead	Caller WAITS for the callee to return before continuing. Like a function call.
Asynchronous Message	Solid line + open arrowhead	Caller does NOT wait. Fire-and-forget. Like sending an email.
Return Message	Dashed line + open arrowhead	The response from the called method back to the caller.
Self-Message	Arrow looping back to same lifeline	An object calling one of its own methods internally.
Creation	Arrow pointing into a new box	One object instantiating another (new). Box appears below the top.
Deletion	X mark at end of lifeline	Object is destroyed / garbage collected. No more messages after X.

2.2 Combined Fragments — Controlling Flow

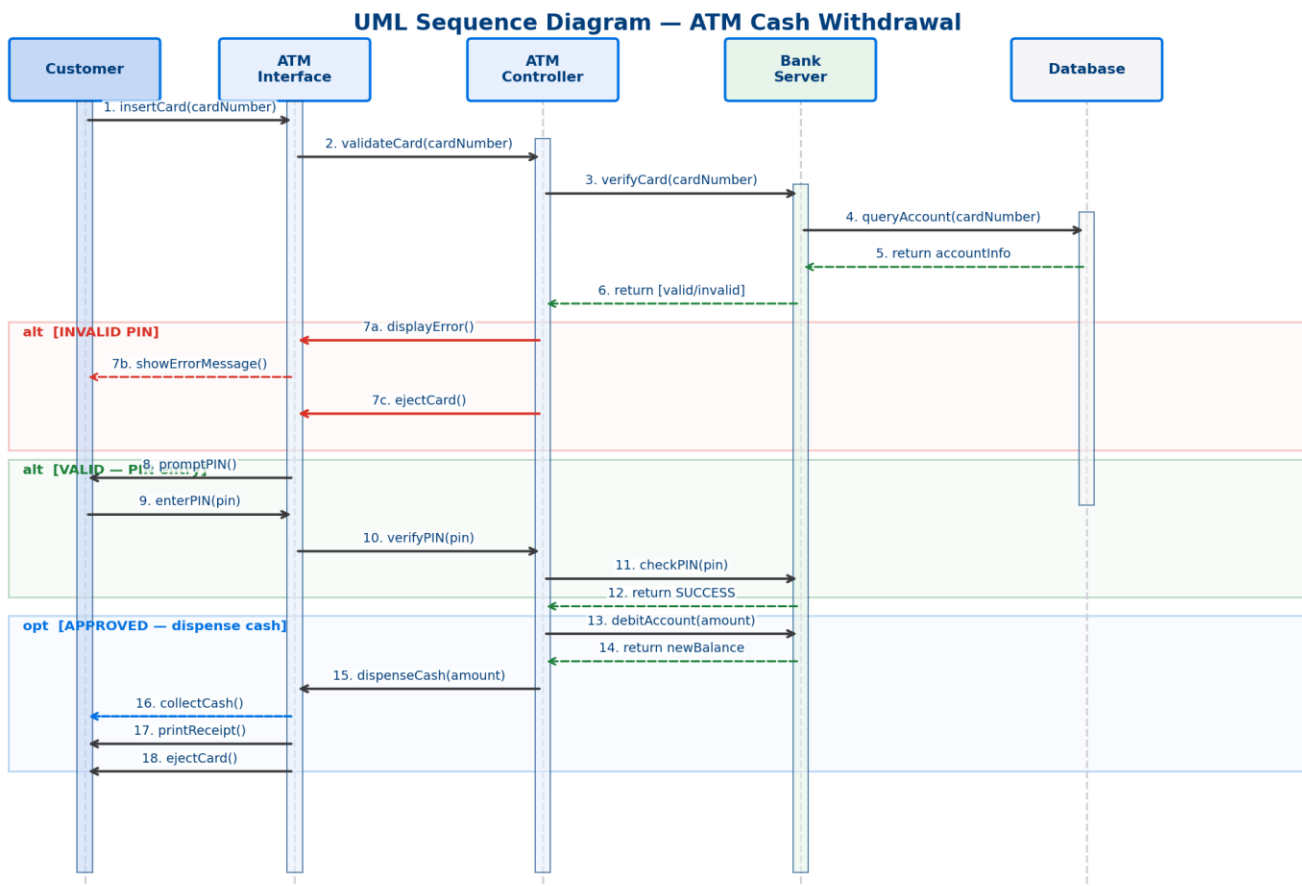
Combined fragments are rectangles drawn around a region of the sequence diagram. They let you express conditions, loops, and parallel execution — things that can't be shown with simple arrows alone.

Operator	Fragment Name	What It Models
alt	Alternative	If-else logic. Multiple operands separated by dashed line. Only ONE executes.
opt	Option	A block that executes only if the guard condition is true. Like 'if' without 'else'.

Operator	Fragment Name	What It Models
loop	Loop	Repeated execution. Can specify loop guard and number of iterations.
par	Parallel	Two or more sequences execute simultaneously (concurrently).
ref	Reference	References another complete sequence diagram by name.
break	Break	Exits the enclosing fragment when its guard is true.
critical	Critical Region	Ensures a sequence cannot be interrupted — like a mutex lock.

2.3 Full Sequence Diagram — ATM Cash Withdrawal

This diagram traces the complete interaction from card insertion to cash dispensing. It shows synchronous messages, return messages, alt fragments, and an opt fragment for the approved flow.



Step-by-Step Trace

1. Customer inserts card. ATMInterface receives cardNumber and sends validateCard() to ATMController synchronously.

2. ATMController calls verifyCard() on BankServer. BankServer queries the Database and receives account info back.
3. BankServer returns [valid/invalid] to ATMController.
4. alt [INVALID]: ATMController tells ATMInterface to display an error and eject the card.
5. alt [VALID]: ATMInterface prompts the Customer to enter their PIN.
6. Customer enters PIN. It is passed through ATMInterface →ATMController →BankServer for verification.
7. opt [APPROVED]: ATMController tells BankServer to debit the account. BankServer returns new balance.
8. ATMController instructs ATMInterface to dispense cash, print a receipt, and eject the card.

EXAM TIP: In a sequence diagram: time flows DOWNWARD. Lifelines run VERTICALLY. Messages are HORIZONTAL arrows. The topmost message is the first event in the scenario. Activation bars show when an object is 'busy'.

Chapter 3 — Coupling and Cohesion

Coupling and Cohesion are the two most fundamental quality metrics in all of software engineering. They were defined by Larry Constantine in the late 1960s and have guided engineers ever since. The golden rule: design systems with HIGH COHESION and LOW COUPLING.

3.1 Cohesion — 'Elements inside a module should belong together'

Cohesion measures how closely the elements INSIDE a single module (class, function, component) are related to each other and work toward one single, clear purpose. High cohesion = focused and purposeful. Low cohesion = scattered and doing too many unrelated things.

Analogy: A kitchen knife is highly cohesive — every part of it serves one purpose: cutting. A 'utility drawer' full of random objects (scissors, tape, matches, coins) has low cohesion — things are grouped only because they had nowhere else to go.

Types of Cohesion — Weakest to Strongest

Type	Grouping Basis	Example
1. Coincidental (WORST)	Completely random	A class with printReport(), calculateTax(), and sendEmail() — no relation at all
2. Logical	Similar category but different purpose	One class handles all I/O: readFile(), readKeyboard(), readNetwork()
3. Temporal	Executed at the same time	Startup class: initDB(), loadConfig(), startLogger() — all run on boot
4. Procedural	Must run in a certain sequence	open() → read() → process() → close() steps in one class
5. Communicational	Operate on the same data	All methods read and update the same user record
6. Sequential	Output of one step feeds the next	parse() result feeds validate(), which feeds save()
7. Functional (BEST)	All elements contribute to one specific task	PasswordHasher: only hash() and verify() — nothing else

3.2 Coupling — 'Modules should be independent of each other'

Coupling measures the degree of interdependence BETWEEN separate modules. High coupling means modules are tightly connected — changing one forces changes in others. Low coupling means modules interact through clean, stable interfaces and are largely independent.

Analogy: Fairy lights in **SERIES** — if one bulb blows, the whole string goes dark (**HIGH COUPLING**). Fairy lights in **PARALLEL** — if one bulb blows, all the others stay lit (**LOW COUPLING**). Parallel wiring = low coupling in electrical design.

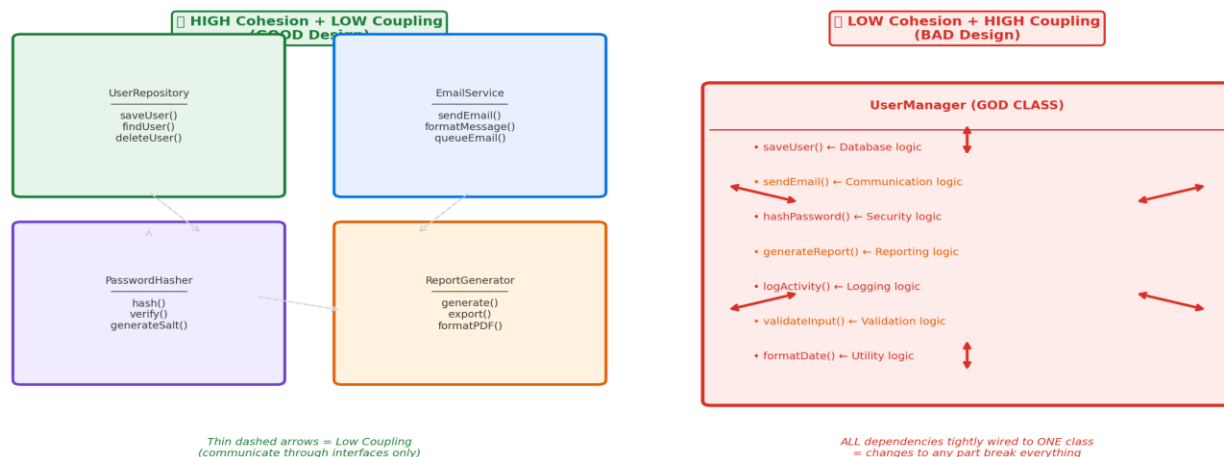
Types of Coupling — Strongest/Worst to Weakest/Best

Type	How Modules Connect	Why It's Harmful / Good
1. Content (WORST)	Module A directly reads/modifies Module B's internal code or data	Completely breaks encapsulation. Any internal change in B destroys A.
2. Common	Multiple modules share a global variable	Change to the global data affects all modules. Race conditions. Impossible to test.
3. External	Modules share an external format, protocol, or device	Changing file format or protocol forces changes in every module that uses it.
4. Control	Module A passes a flag telling Module B how to behave	A must know B's internals to set the flag correctly. Creates hidden dependency.
5. Stamp	A full data structure is passed but only part is needed	Changes to unused fields of the structure still require testing of the consumer.
6. Data (BEST)	Modules communicate only via simple, primitive parameters	Clean boundary. Each module is self-contained. Easiest to test and maintain.

3.3 Visual Comparison — Good Design vs. Bad Design

The diagram below shows the difference side by side. On the left: four cohesive classes with low coupling through interfaces. On the right: one 'God class' with all responsibilities jammed in and tight coupling everywhere.

Coupling & Cohesion — Good Design vs. Bad Design



3.4 Code Example — Before and After

BEFORE (Low Cohesion + High Coupling)

```
class UserManager {
    saveUser(user)          { /* database SQL */ }
    sendWelcomeEmail(user)  { /* SMTP email  */ }
    generateInvoice(user)   { /* PDF render   */ }
    hashPassword(password) { /* BCrypt hash  */ }
    logUserAction(user, action) { /* log file */ }
}
// Problem: 5 completely different responsibilities.
// Any change to email, PDF, DB, or hashing forces this class to change.
```

AFTER (High Cohesion + Low Coupling)

```
class UserRepository { saveUser(user) { ... } }
class EmailService   { sendWelcomeEmail(user) { ... } }
class InvoiceGenerator { generateInvoice(user) { ... } }
class PasswordHasher { hash(pw) { ... } verify(pw, hash) { ... } }
class ActivityLogger { log(user, action) { ... } }

// Each class has exactly ONE reason to change.
// They communicate through constructor injection (low coupling).
// Each can be tested completely independently.
```

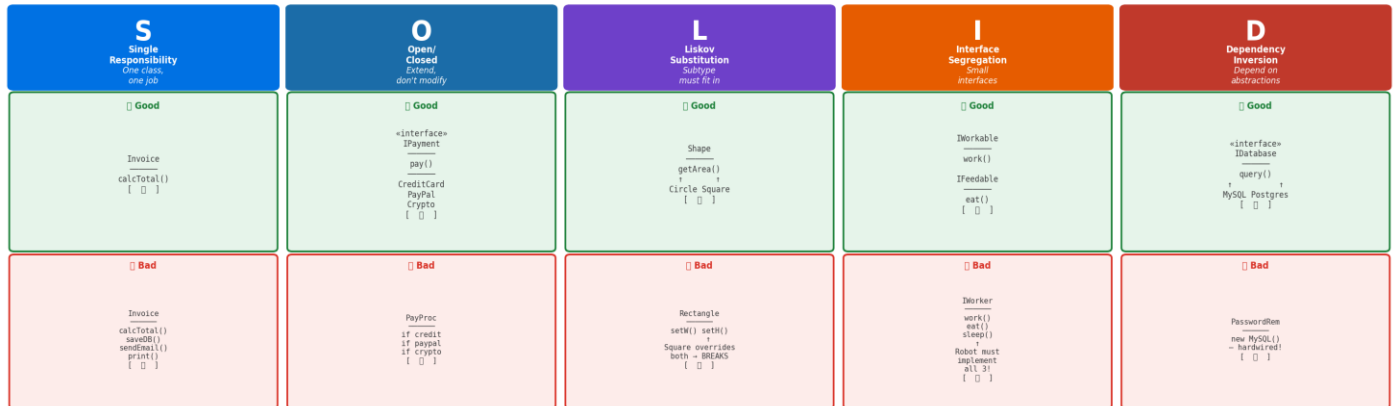
EXAM TIP: Cohesion is INTRA-module (inside one module). Coupling is INTER-module (between modules). High cohesion naturally CAUSES low coupling — a focused class needs to interact with fewer other modules. They reinforce each other.

Chapter 4 — The SOLID Principles

SOLID is an acronym for five object-oriented design principles introduced by Robert C. Martin (known as Uncle Bob) in his year-2000 paper 'Design Principles and Design Patterns'. The acronym was coined by Michael Feathers around 2004. Together, these five principles form the foundation of clean, maintainable, and scalable software architecture.

SOLID: S = Single Responsibility | O = Open/Closed | L = Liskov Substitution | I = Interface Segregation | D = Dependency Inversion

The SOLID Principles — Visual Overview



S — Single Responsibility Principle (SRP)

"A class should have one, and only one, reason to change."

Every class should be responsible for a single piece of functionality. If a class handles multiple responsibilities, a change to any one of them risks breaking all the others. The goal is not necessarily to make very small classes, but to ensure each class has one clearly defined purpose.

Real-World Analogy: In a hospital, a surgeon operates. A nurse handles patient care. A pharmacist manages medication. An administrator handles billing. Each person has a single responsibility. If the surgeon also had to do billing, a change in billing procedures would disrupt surgical operations.

Violation — One Class Doing Everything

```
class Order {  
    calculateTotal() { /* Business logic */ }  
    saveToDatabase() { /* Data persistence */ }  
    sendConfirmEmail() { /* Communication */ }  
    printInvoicePDF() { /* Presentation */ }  
}  
// 4 reasons to change: business rules, DB schema, email system, PDF format.
```

Solution — Separate Responsibilities

```
class Order { calculateTotal() { ... } }  
class OrderRepository { saveToDatabase(order) { ... } }
```

```
class OrderEmailer { sendConfirmEmail(order) { ... } }
class InvoicePrinter { printInvoicePDF(order) { ... } }
```

O — Open/Closed Principle (OCP)

"Software entities should be open for extension, but closed for modification."

You should be able to add new features to a system WITHOUT touching (and potentially breaking) already-tested existing code. This is achieved through abstraction: define interfaces or abstract classes, and add new behaviour by creating new implementing classes.

Real-World Analogy: A power socket is closed for modification (you never rewire the wall). But it is open for extension — any device with the right plug can use it (phone, laptop, fan, kettle). New devices can be added without changing the wall.

Violation — Adding PayPal means editing PaymentProcessor

```
class PaymentProcessor {
  process(type, amount) {
    if (type == 'credit') { /* credit card logic */ }
    else if (type == 'paypal') { /* paypal logic */ }
    // Adding crypto? MUST modify this class. Risky!
  }
}
```

Solution — New payment type = new class, zero modifications

```
interface IPayment { pay(amount); }

class CreditCardPayment implements IPayment { pay(a) { /*card*/ } }
class PayPalPayment implements IPayment { pay(a) { /*paypal*/ } }
class CryptoPayment implements IPayment { pay(a) { /*crypto*/ } }

class PaymentProcessor {
  process(IPayment method, amount) { method.pay(amount); }
}
// Add crypto? Create CryptoPayment. PaymentProcessor NEVER changes.
```

L — Liskov Substitution Principle (LSP)

"Objects of a superclass should be replaceable with objects of a subclass without altering the correctness of the program."

Any code that works with a base class must work equally correctly when given a subclass. Subclasses must honour the BEHAVIOURAL CONTRACT of the parent class — they can extend it, but must not weaken it or change expected outcomes.

Real-World Analogy: If you are driving a Car, you should be able to replace it with any other type of Vehicle (Truck, Van, Motorcycle) and still be able to drive it. If one 'Vehicle' required you to pull a rope instead of turn a steering wheel, it violates LSP.

Classic LSP Violation — The Rectangle/Square Problem

```
class Rectangle {
    setWidth(w) { this.width = w; }
    setHeight(h) { this.height = h; }
    getArea() { return this.width * this.height; }
}

class Square extends Rectangle {
    setWidth(w) { this.width = w; this.height = w; } // Forces equal
    setHeight(h) { this.width = h; this.height = h; } // Forces equal
}

// Client code that works for Rectangle:
r.setWidth(5); r.setHeight(4);
// Expected: area = 20. Actual with Square: area = 16. BROKEN.
```

Solution — Don't force Square to inherit Rectangle

```
interface IShape { getArea(); }
class Rectangle implements IShape { setWidth(w); setHeight(h); getArea() { return w*h; } }
class Square implements IShape { setSide(s); getArea() { return s*s; } }
// Both honour the IShape contract. Neither breaks the other.
```

I — Interface Segregation Principle (ISP)

"Clients should not be forced to depend on interfaces they do not use."

One large, bloated interface forces implementing classes to provide methods they have no use for (typically throwing 'not supported' exceptions). Instead, break large interfaces into smaller, focused ones. A class should only need to implement the methods relevant to it.

Real-World Analogy: A universal remote control with 200 buttons is overwhelming and most are irrelevant for any given device. ISP says: give each device a minimal remote with only the buttons it needs. A TV remote needs volume and channels, not a washing machine spin cycle.

Violation — Robot is forced to implement eat() and sleep()

```
interface IWorker {
    work();
    eat(); // Robots do not eat!
    sleep(); // Robots do not sleep!
}

class RobotWorker implements IWorker {
    work() { /* does work */ }
    eat() { throw new Error('Robots do not eat!'); } // Forced garbage
    sleep() { throw new Error('Robots do not sleep!'); } // Forced garbage
}
```

Solution — Small, focused interfaces

```
interface IWorkable { work(); }
interface IFeedable { eat(); }
interface ISleepable { sleep(); }

class HumanWorker implements IWorkable, IFeedable, ISleepable { ... }
class RobotWorker implements IWorkable { work() { /* only this */ } }
```

D — Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions."

Business logic (high-level) should not be hardwired to specific implementations (low-level). Instead, both should communicate through stable interfaces. This allows you to swap implementations (e.g., change database, change payment provider) without touching your business logic.

Real-World Analogy: Your laptop charger plugs into a standard power socket (abstraction). The laptop does not hardwire directly into the building's electrical circuit (concrete). If you travel internationally, you use an adapter — you never modify the laptop itself.

Violation — PasswordReminder is hardwired to MySQL

```
class PasswordReminder {
    constructor() {
        this.db = new MySQLConnection(); // Tightly coupled!
    }
    remind(email) { this.db.query('SELECT ...'); }
}
// Switch to PostgreSQL? You MUST edit PasswordReminder. That's wrong.
```

Solution — Inject the dependency through an abstraction

```
interface IDatabase { connect(); query(sql); }

class MySQLConnection implements IDatabase { connect(){} query(s){} }
class PostgreSQLConnection implements IDatabase { connect(){} query(s){} }

class PasswordReminder {
    constructor(IDatabase db) { // Injected from outside
        this.db = db;
    }
    remind(email) { this.db.query('SELECT ...'); }
}
// Switch database? Just inject a different IDatabase. Zero changes to business logic.
```

4.6 SOLID Summary Table

	Principle	Core Statement	Achieved By
S	Single Responsibility	One class, one job, one reason to change	Split unrelated methods into separate classes

	Principle	Core Statement	Achieved By
O	Open/Closed	Extend don't modify existing code	Use interfaces/abstract classes + new subclasses
L	Liskov Substitution	Subtype must be safely substitutable for parent	Ensure subclasses honour parent's contract
I	Interface Segregation	Many small interfaces, not one fat interface	Create role-specific interfaces for each client
D	Dependency Inversion	Depend on abstractions, not concretions	Inject dependencies via constructor or methods

EXAM TIP: SRP is the software version of High Cohesion. DIP is the design mechanism that enforces Low Coupling. Both SOLID and Cohesion/Coupling lead to the same ideal: focused, independent, testable modules.

Chapter 5 — UML Activity Diagram

An Activity Diagram is a behavioral UML diagram that models workflow, processes, and sequential (or parallel) activities in a system. It is essentially an advanced flowchart that additionally supports concurrent flows, swim lanes (assigning who does what), and object flows (tracking data between activities).

Core question answered: What are the steps, in what order, who performs them, and can any steps happen simultaneously?

5.1 Core Symbols and Notation

Symbol / Name	Shape	Description
Initial Node	Filled black circle	The single starting point of the entire activity flow
Activity Final Node	Circle within circle	The end point that terminates the ENTIRE activity
Flow Final Node	Circle with X inside	Terminates ONE flow but not the entire activity
Action / Activity	Rounded rectangle	A single step, task, or action being performed
Decision Node	Diamond shape	A branch point: one input, multiple guarded outputs. Like an if-statement.
Merge Node	Diamond shape	Multiple paths converge back into a single flow
Fork Node	Thick horizontal/vertical bar	ONE flow splits into MULTIPLE parallel flows (concurrent execution)
Join Node	Thick horizontal/vertical bar	Multiple concurrent flows SYNCHRONISE back into ONE (waits for all)
Swim Lane / Partition	Vertical dashed line boundary	Divides the diagram by actor/system. Shows WHO is responsible for each activity.
Guard Condition	Text in [square brackets]	A condition that must be true for a transition to be taken

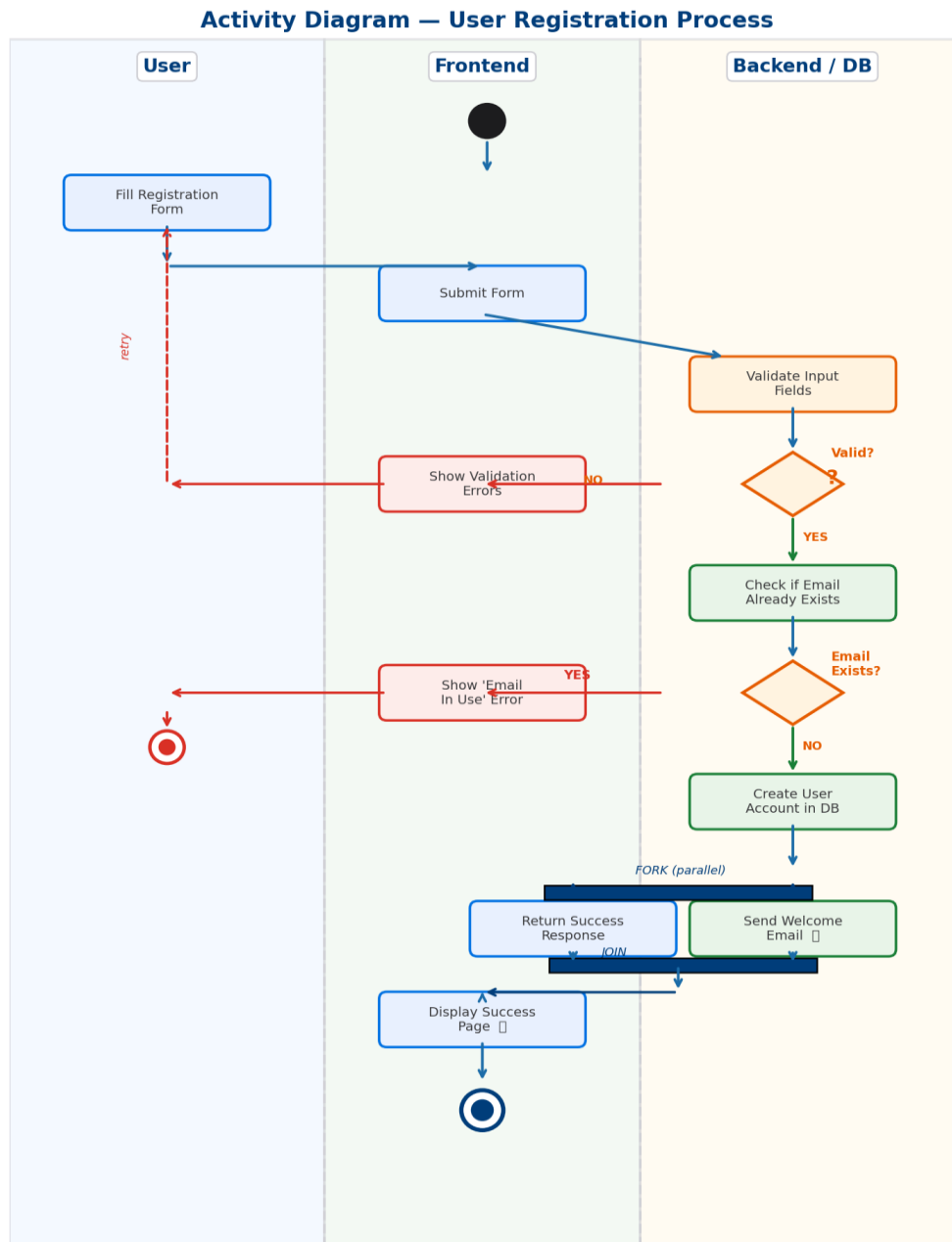
5.2 Swim Lanes — Assigning Responsibility

Swim lanes (also called partitions) are the most powerful feature of activity diagrams. They divide the diagram into vertical columns, one per actor or system component. Each action is placed inside the lane of whoever performs it. This makes it immediately obvious who is responsible for each step.

Example Swim Lanes for an Online Purchase: Customer | Website Frontend | Payment Gateway | Warehouse | Delivery Service. 'Add to Cart' lives in Customer. 'Process Payment' lives in Payment Gateway. 'Pack Order' lives in Warehouse.

5.3 Full Activity Diagram — User Registration

This diagram models the complete user registration flow for a web application. It uses three swim lanes (User, Frontend, Backend/DB), decision nodes, a fork for parallel execution, and a join node.



Key Elements to Note in This Diagram

- Initial Node (top) — the process begins when the user opens the registration page.
- Decision Node 'Valid?' — If validation fails, errors are shown and the user returns to the form (loop back).
- Decision Node 'Email Exists?' — Catches duplicate registrations before any account is created.
- FORK bar — After creating the account, two actions happen IN PARALLEL: sending the email AND returning the HTTP response. Both happen simultaneously.
- JOIN bar — The flow waits for BOTH parallel branches to complete before proceeding to show the success page.

- Activity Final Node (bottom) — the process ends successfully.

EXAM TIP: Fork = one flow becomes many (split). Join = many flows become one (synchronise). Use Fork/Join for truly parallel operations (e.g., sending email + logging simultaneously). Decision/Merge are for conditional branches — only ONE path executes.

Chapter 6 — State Machine Diagram

A State Machine Diagram (also called a Statechart or State Diagram) models the dynamic lifecycle of a SINGLE OBJECT. It shows every state the object can be in, and the events or conditions that trigger transitions between states.

The defining characteristic: the same event can produce DIFFERENT outcomes depending on the current STATE of the object. A locked door ignores the 'open' command. An unlocked door obeys it. Same event — different result — because of state.

6.1 Core Components

Component	Symbol	Description
Initial Pseudostate	Filled black circle	The starting point. The object enters its first real state from here.
Final State	Concentric circles	The object's lifecycle has ended. Optional — some objects never terminate.
State	Rounded rectangle	A stable condition the object is in (e.g., Placed, Shipped, Delivered).
Transition	Arrow between states	Movement from one state to another, triggered by an event.
Entry Action	'entry/' inside state	An action executed EVERY TIME the object enters this state.
Exit Action	'exit/' inside state	An action executed EVERY TIME the object leaves this state.
Do Activity	'do/' inside state	An ongoing activity that runs for the ENTIRE DURATION of the state.
Guard Condition	[condition] on arrow	A boolean expression that must be TRUE for the transition to fire.
Composite State	State with nested states	A state that contains sub-states (for modelling complex behaviours).
Choice Pseudostate	Diamond shape	Dynamic branching based on a runtime condition.

6.2 Transition Notation — The Full Format

Every arrow (transition) in a state machine diagram can optionally carry a label in this precise format:

event [guard condition] / action

Part	Required?	Meaning
event	Optional	The external trigger that causes this transition to be considered (e.g., insertCoin, timeout, buttonPress)

Part	Required?	Meaning
[guard]	Optional	A condition in square brackets that must be TRUE at the moment of the event for this transition to fire
/ action	Optional	An action executed atomically during the transition itself, before entering the next state

Example transition label: `pressButton [doorsClosed] / lockDoor`

Reading: When 'pressButton' event fires AND the guard 'doorsClosed' is TRUE, execute the action 'lockDoor', then transition to the Locked state.

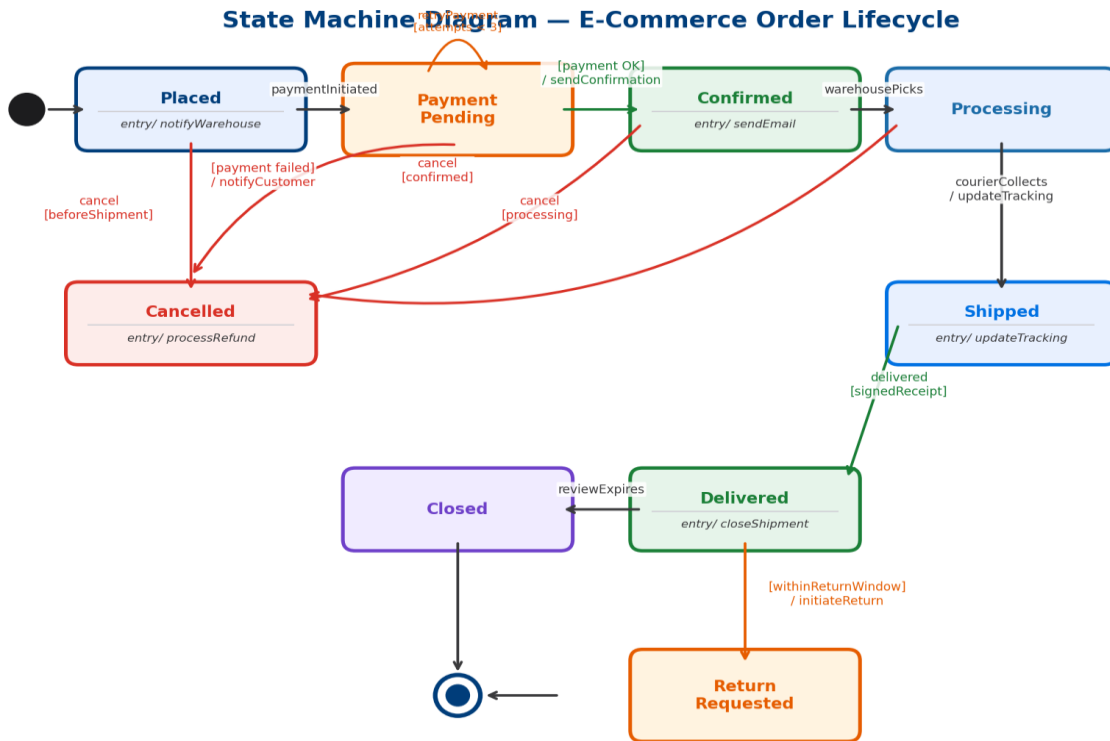
6.3 Real-World Example: Vending Machine States

From State	Event	Guard / Action	To State
Idle	selectItem	[item in stock] / displayPrice	ItemSelected
Idle	selectItem	[item out of stock] / showMessage	OutOfStock
ItemSelected	insertCoin	/ updateBalance	PaymentPending
PaymentPending	insertCoin	[balance < price] / updateBalance	PaymentPending (self)
PaymentPending	insertCoin	[balance >= price] / dispenseChange	DispensingItem
PaymentPending	cancel	/ returnAllCoins	Idle
DispensingItem	itemDispensed	/ updateInventory	Idle
OutOfStock	restockItem	/ updateInventory	Idle

6.4 Full State Machine Diagram — E-Commerce Order Lifecycle

This is a classic and complete exam example. It shows every state an Order object can be in from the moment it is placed until it is closed or returned.

State Machine Diagram — E-Commerce Order Lifecycle



Reading This Diagram

- Initial node —Placed: Every order starts in the Placed state. Entry action: notify warehouse.
- Placed —PaymentPending: triggered by the 'paymentInitiated' event.
- PaymentPending —Confirmed: guard [payment OK] must be true. Action: send confirmation email.
- PaymentPending —Cancelled: guard [payment failed]. Action: notify customer.
- Self-loop on PaymentPending: 'retryPayment [attempts < 3]' — the customer can retry up to 3 times.
- Confirmed —Processing —Shipped —Delivered —Closed: the normal happy path.
- Cancellation transitions: From Placed, Confirmed, or Processing —Cancelled (before shipment only).
- Delivered —ReturnRequested: only possible within the return window. Then —Final state.

6.5 Activity Diagram vs. State Machine — Side by Side

Feature	Activity Diagram	State Machine Diagram
Models	Workflow / business process	Lifecycle of a single object
Nodes represent	Actions (things being done)	States (conditions the object is in)
Transitions triggered by	Completion of an activity	Events, conditions, or time
Parallel execution	Fork and Join bars	Concurrent sub-states (regions)

Feature	Activity Diagram	State Machine Diagram
Responsibility shown	Swim lanes (who does what)	Not applicable — single object
Best used for	Use cases, system processes, workflows	Objects with complex event-driven lifecycles
Classic example	Online checkout process	Order: Placed → Confirmed → Shipped → Delivered

EXAM TIP: State Machine Diagrams excel at showing that the **SAME EVENT** causes **DIFFERENT** outcomes based on the **CURRENT STATE**. 'cancel' from Placed is allowed. 'cancel' from Shipped is not. This state-dependent behaviour cannot be captured cleanly in a flowchart.

Chapter 7 — Exam Cheat Sheet

Condensed revision of every examinable point — sorted by topic.

Class Diagram

- **Six relationships: Association, Aggregation, Composition, Inheritance, Realization, Dependency**
- Filled diamond = Composition (child DIES with parent) | Hollow diamond = Aggregation (child SURVIVES)
- Hollow triangle arrowhead = Inheritance (is-a) | Dashed + hollow triangle = Realization (implements)
- Dashed arrow only = Dependency (temporarily uses) | Plain solid line = Association (interacts)
- Visibility: + public, - private, # protected, ~ package | Multiplicity: 1, 0..1, *, 0..*, 1..*
- Three compartments: Class Name | Attributes (data) | Operations (methods)

Sequence Diagram

- **Time flows TOP to BOTTOM. First message at top, last at bottom.**
- Lifeline = vertical dashed line. Activation bar = thin rectangle on lifeline (object is executing)
- Solid filled arrow = synchronous call (caller waits) | Dashed return arrow = response
- alt = if-else | opt = if only | loop = repeat | par = concurrent execution
- Self-message = object calling its own method | X on lifeline = object destroyed
- Sequence diagram captures ONE SPECIFIC SCENARIO (not all possible scenarios)

Coupling & Cohesion

- **TARGET: HIGH Cohesion + LOW Coupling.**
- Cohesion = how related elements INSIDE a module are (intra-module)
- Coupling = how dependent modules are ON EACH OTHER (inter-module)
- Best cohesion = Functional (one specific task) | Worst = Coincidental (random grouping)
- Best coupling = Data (params only) | Worst = Content (directly accessing internals)
- High cohesion naturally produces low coupling — they are inversely correlated in bad designs

SOLID Principles

- **S = SRP: one class, one reason to change. Split when class does unrelated things.**
- **O = OCP: extend via new classes, never modify working tested code. Use interfaces.**
- **L = LSP: every subclass must behave correctly wherever the parent is expected. No broken substitutions.**
- **I = ISP: many small interfaces beat one fat interface. Classes only implement what they need.**
- **D = DIP: business logic (high-level) depends on interfaces, not concrete implementations (low-level).**
- DIP enables Dependency Injection: inject dependencies from outside via constructor or setter.

Activity Diagram

- **Models WORKFLOW — what activities happen, in what order, and who does them.**
- Initial = filled circle | Final (activity) = circle within circle | Final (flow) = circle with X
- Decision = diamond (branch, conditional) | Merge = diamond (paths rejoin, no condition)
- Fork = thick bar (one → many parallel paths) | Join = thick bar (many → one, WAITS for all)
- Guard conditions are written in [square brackets] on transitions
- Swim Lanes (partitions) show which actor/system is responsible for each activity

State Machine Diagram

- **Models the LIFECYCLE of a SINGLE OBJECT and its responses to events.**
- State = stable condition of the object (rounded rectangle)
- Transition label format: event [guard] / action — all three parts are optional
- Initial pseudostate = filled circle | Final state = concentric circles
- Entry action = runs on entering state | Exit action = runs on leaving state | Do = runs during state
- KEY INSIGHT: Same event → different transition depending on CURRENT STATE
- Classic state machine examples: Door (Open/Closed/Locked), Traffic Light, ATM, Order lifecycle

Good luck on your exam, Danial. You have everything you need!

Read every chapter. Study every diagram. Review this cheat sheet the night before.